

ПРАКТИЧЕСКИЙ УЧЕБНИК

АГЕНТНЫЕ ПАЙПЛАЙНЫ

Контроль, верификация
и надёжность разработки

Роли / Хуки / Проверки / Конечные автоматы

Антон

Исходный авторский текст

Астра

Комментарии, пояснения и планы внедрения

ИЗДАНИЕ 1 · 12 СЕНТЯБРЯ 2026

Об издании

Исходный текст: Антон. Автор комментариев, пояснений терминов, учебных примеров, этапов реализации и заключительного разбора: Астра, ИИ-ассистент.

Материал предназначен для читателей, которые хотят понять устройство управляемой агентной разработки. Специальные термины объясняются по ходу глав; базового представления о программе, файлах и проверке результата достаточно для первого чтения.

Как сохранён авторский текст

Содержательные авторские формулировки сохранены без переписывания. Удалены разговорная вставка о просьбе к агенту подготовить материал, служебная фраза о сборе матрицы и повторный экземпляр двух блоков — о принципах контроля с теорией надёжности и о конечном автомате. Разделы разбиты на главы; пробелы, переносы и оформление списков нормализованы.

Авторская оценка рынка и технические утверждения воспроизведены в исходном виде. Если они требуют уточнения или обновились, комментарий помещён отдельно и подписан Астрой. Приведённые в исходнике количества правил и хуков не являются результатом аудита в рамках этого издания.

Как читать

Главы 1–6 начинаются авторским текстом. Затем идут пояснения Астры и практический план внедрения. Глава 7 целиком написана Астрой: в ней предложены дополнительные механизмы, критерии измерения и дорожная карта. В приложении — шаблоны артефактов и вопросы для самопроверки.

Оформление	Авторство
Основной текст с засечками	Авторский материал Антона.
Разделы с заголовком «Астра» и шрифт без засечек	Пояснения, инженерные предложения и аналитика Астры.
Ссылки [1]–[12]	Проверенные первоисточники для дополнений Астры. Ссылки активны в PDF.

Дата проверки внешних материалов: 12 сентября 2026 года. Предложения по реализации — рекомендации для проектирования; их готовность в конкретном проекте подтверждается испытаниями.

Оглавление

Нажмите на название раздела или используйте закладки PDF.

1. Агентный пайплайн: основной цикл	4
Астра: пояснения к главе	5
Астра: этапы реализации	6
2. Матрица хуков: как правила дублируются кодом	8
Астра: пояснения к главе	9
Астра: этапы реализации	11
3. Фундамент: принципы контроля работы агентов	12
Астра: пояснения к главе	12
Астра: этапы реализации	13
4. Переключка с теорией надёжности	15
Астра: пояснения к главе	16
Астра: этапы реализации	17
5. Пайплайн как конечный автомат	18
Астра: пояснения к главе	19
Астра: этапы реализации	20
6. Пайплайн против рынка: объективная сверка	22
Астра: пояснения к главе	23
Астра: этапы реализации	24
7. Разбор Астры: от дисциплины агентов к проверяемой системе	26
Астра: дорожная карта внедрения	29
Приложение А. Шаблоны рабочих артефактов	31
Источники и дальнейшее чтение	33

ГЛАВА 01

1. Агентный пайплайн: основной цикл

АВТОРСКИЙ ТЕКСТ · АНТОН

Роли.

Главный агент (оркестратор) кода не пишет — ни одной правки, включая «мелочи». Он формулирует задачи, запускает исполнителей, собирает вердикты и коммитит. Код и тесты пишут воркеры-сабагенты, проверяют — отдельные read-only верификаторы. Автор никогда не проверяет сам себя.

Цикл: бриф → воркеры → верификация → loop

1. Бриф. Перед каждым запуском отдельный агент-брифер готовит самодостаточный пакет: объём, список файлов, выдержки из спеки, правила, критерии приёмки — а при генерации кода ещё и предопределённый контракт имён (testid'ы, API, i18n-ключи, пропсы). Воркер получает всё в промпте, а не ищет по репозиторию.
2. Воркеры. При генерации нового модуля — два агента параллельно: тест-агент пишет поведенческие тесты из брифа, не видя кода; код-агент пишет реализацию, не видя тестов. Тесты — исполняемая спецификация: тест-агент не заражён реализацией, реализация не подгоняется под собственные тесты. Расхождение имён предотвращено контрактом из брифа; остаток ловит верификатор. При багфиксе схема строже — TDD-последовательность: сначала тест-агент (падающий тест), затем отдельный фикс-агент, потом верификатор.
3. Верификаторы. Каждый вывод воркера проверяют read-only агенты, работающие параллельно:
 - Спек-верификатор сверяет дифф с брифом и спекой: точность, полнота 1:1, отсутствие выдуманного, отсутствие регрессий. Итог — список замечаний + вердикт PASS/FAIL.
 - Адверсальный подключается, когда дифф касается денег/платежей/безопасности. Задача — сломать код конкретным сценарием («что реально произойдёт с балансом пользователя при выполнении этого payload»), а не перечитать спеку. Пройти без предъявленного сценария нельзя.
4. Loop. FAIL → новый воркер, которому верификатор передал замечания через файл — оркестратор руками ничего не пересылает. Бюджет — максимум 3 итерации, дальше эскалация к человеку с сохранённым диффом. Бесконечных циклов нет.

Правила, которые держат систему

- Оркестратор не кодит — иначе перестаёт быть независимым контролёром.
- Нет брифа — нет диспетча: воркер не запускается без полного пакета.
- Нет неверифицированного вывода: любой результат проходит верификатора.

- Модель по роли: сильная — брифам и верификаторам, дешёвая и быстрая — коду; вручную модель не выбирают.
- Каждая находка маршрутизируется: дефект в своём диффе правится немедленно; в чужом/старом коде — эскалируется владельцу списком. Ничего не «теряется».
- Гейт на неоднозначность: неполная спека → вопрос человеку до запуска воркера, а не молчаливый стаб.
- Хуки-бэкстопы: ключевые правила дублируются детерминированными shell-хуками (запрет оркестратору писать код, обязательный верификатор, бюджет итераций, таймаут диспетча). LLM-инструкция — слабая гарантия; правило без хука считается незащищённым.

Параллельная запись

Read-only агенты (верификаторы, аналитики) работают параллельно всегда. Мутации — в трёх формах: спаренная пара «тест + код» в одном дереве (с запретом читать друг друга); независимые модули — изолированными git-worktree-цепочками бриф → воркер → верификатор → merge, где merge + typescheck работают как мьютекс, а волна закрывается одним коммитом; всё остальное — строго последовательно.

Астра: пояснения к главе

Пайплайн — последовательность работ с условиями перехода между ними. Здесь результатом служит проверенное изменение программы. LLM — большая языковая модель; агент — модель вместе с инструментами, контекстом и правилами выполнения. Разделение на роли распределяет права и информацию, но само по себе не доказывает независимость ошибок.

Термин	Простое объяснение
Оркестратор; воркер; сабагент	Координатор; исполнитель; отдельный агент, запущенный для подзадачи. Один и тот же тип модели может обслуживать разные роли.
Бриф; спека; контракт	Пакет конкретной задачи; описание требуемого поведения; точные договорённости об интерфейсах и ограничениях.
Промпт; контекст; репозиторий	Инструкция модели; вся доступная ей информация; хранилище файлов проекта и истории изменений.
API; пропсы; i18n-ключи; testid	Интерфейс вызова программы; входные параметры компонента; идентификаторы переводимых строк; метки элементов для тестов.
Поведенческий тест; TDD	Проверка наблюдаемого поведения; разработка через сначала падающий тест, затем исправление и последующее улучшение кода.
Дифф; регрессия; PASS/FAIL	Набор изменений; поломка ранее работавшего поведения; положительный или отрицательный вердикт.

Термин	Простое объяснение
Read-only; мутация; payload	Доступ без изменения проверяемого объекта; изменение состояния; конкретные входные данные запроса.
Диспетч; loop; rework; эскалация	Запуск исполнителя; повтор цикла; доработка; передача решения человеку или ответственному владельцу.
Worktree; merge; коммит; мьютекс	Отдельная рабочая копия Git; объединение изменений; сохранённая версия; механизм, допускающий к общему ресурсу одного участника.

Астра: что нужно уточнить до запуска

Независимая приёмка совместима с самопроверкой исполнителя. Запуск компилятора или тестов автором помогает найти ошибки; право окончательно принять результат остаётся у другого участника. Если понимать «автор никогда не проверяет сам себя» как запрет любой локальной проверки, цикл станет дороже без автоматического выигрыша в надёжности. Это предлагаемое уточнение процесса, а не исправление авторского текста.

Контракт имён устраняет только расхождения в названиях. Для одинакового понимания задачи нужны ещё типы, обязательные поля, коды ошибок, единицы измерения и правила поведения на границах. Например, имя `calculateFee` не отвечает на вопросы о валюте, округлении и отрицательной сумме.

Самодостаточный бриф должен ссылаться на определённую версию проекта. Полезен контролируемый запрос недостающего контекста через брифера: запрет на самостоятельные догадки не должен превращаться в обязанность работать с неполной информацией. Верификатору нужен доступ к исходной спецификации, иначе ошибка её пересказа попадёт во все роли сразу.

В одном дереве запрет читать работу соседа остаётся условностью, пока среда технически разрешает такое чтение. Для строгого эксперимента «вслепую» я предпочла бы отдельные снимки исходной версии и передачу результатов доверенному сборщику. Это отличается от предложенной автором общей рабочей копии, но лучше обеспечивает именно информационную изоляцию.

Адверсальный сценарий не обязан находить реальный дефект: иначе исправный код невозможно принять. Обязательны попытка, ожидаемый инвариант и наблюдаемый результат. Например: два одновременных запроса с одним идентификатором операции должны оставить в тестовом журнале одну операцию, а не две. Сценарий, который лишь описан, нельзя выдавать за выполненный.

Астра: этапы реализации

1. Зафиксировать полномочия ролей

Создать таблицу «роль → чтение → запись → инструменты → выпуск результата». Выделить запись отчётов отдельно от изменения продукта: верификатор читает код, но может сохранять свой вердикт в разрешённое место. Результат этапа — права, которые можно проверить попыткой запрещённого действия.

2. Ввести единый формат брифа

Обязательные поля: `task_id`, исходный `commit`, версия спецификации, цель, границы, разрешённые файлы, контракты, критерии приёмки, команды проверки, ограничения и нерешённые вопросы. Диспетчер проверяет структуру программно; брифер проверяет смысл. Пустое поле и противоречивое требование — разные причины отказа.

3. Запустить одну задачу последовательно

Начать с небольшой функции без внешних побочных эффектов. Сохранить бриф, дифф, отчёт проверки и итоговый статус. При багфиксе отдельный проверяющий должен подтвердить: исходный тест падает из-за заявленного поведения, а не из-за отсутствующего импорта или сломанной среды.

4. Привязать приёмку к версии результата

В вердикт включить идентификатор проверенного снимка, критерии, доказательства и находки. После любой содержательной правки предыдущий PASS теряет силу. Приёмка завершена, когда обязательные критерии подтверждены для того же снимка, который готовится к коммиту.

5. Проверить отказ и исчерпание бюджета

Заранее определить смысл трёх итераций. Для первого внедрения я предлагаю максимум три попытки исполнения всего: первая и две доработки. Счётчик хранит управляющий процесс; смена агента его не сбрасывает. Учебный прогон должен показать остановку, сохранение изменений и понятный запрос человеку.

6. Добавить параллелизм после устойчивого прохода

Сначала распараллелить чтение и независимые проверки неизменяемого снимка. Затем испытать пару «тест + код», и лишь после этого — независимые модули. Критерий продвижения: успешная интеграция без ручного переписывания координатором и без потери результатов отдельных исполнителей.

ГЛАВА 02

2. Матрица хуков: как правила дублируются кодом

АВТОРСКИЙ ТЕКСТ · АНТОН

Принцип.

Каждое LLM-правило пайплайна сопоставлено хуку — детерминированному shell-скрипту, который срабатывает на событие до/после вызова инструмента или при коммите. Матрица — живой документ вида RULE → HOOK → ENFORCEMENT: ~60 правил, ~53 хука (плюс общие библиотеки и один хелпер). Правило без хука — это не молчаливый пропуск, а видимая строка «coverage gap».

Три класса исполнения

- Blocking (~40%): хук возвращает deny — действие запрещено физически. Примеры: оркестратору нельзя писать в src/**/tests/** вообще (включая spell-обходы через sed -i, redirect, tee); сабагенту нельзя делать мутации git и диспетчи кроме read-only Explore; нельзя sleep; typecheck/eslint/секреты/сквозные ссылки не прошли — коммит заблокирован; диспетчеру 60 минут — дальше инструментные вызовы прекращаются.
- Soft (~30%): действие разрешено, но предупреждение всплывает в контекст агента (наблюдения после возврата воркера, GAP-эскалация, бюджет итераций, дублирование тестов, дрейф рабочего дерева). Ключевое правило честности: advisory — не гейт: останавливает работу только deny; предупреждение взвешивается против бюджета, и тот, кто его перебивает, обязан назвать, какой бюджет управляет.
- n/a (~30%): детерминированной проверки нет — правило держится только на инструкции. Матрица это фиксирует открыто, а не прячет.

Какие классы правил накрыты

1. Роли и границы — оркестратор не кодит; воркер не делает git/коммиты/чужие диспетчи; модель определяется ролью, а не вручную; каждый диспетч открывается ROLE-маркером.
2. Цепочка верификации — воркер без верификатора не пройдёт; вердикт записывается детерминированно; объём на одного верификатора ограничен (иначе — сплит на параллельных).
3. Процессные гейты — план обязан содержать шаги пайплайна; GAP эскалируется до запуска воркера; после каждого возврата агента из него извлекаются наблюдения.
4. Гигиена Bash — деструктивные команды; запрет sleep; ломалка циклов чтения (третий одинаковый Read без изменений — отказ); мутации git обязаны называть дерево; работа сохраняется как артефакт до любого revert.

5. Коммитные гейты — `typescheck` по всем проектам, `ESLint`, секреты в логах, `pipeline-log` запись, резолв ссылок между документами, «эфемерные» ссылки пайплайна в комментариях.

6. Волны и `worktree` — изоляция мутирующих диспетчей; диспетч называет дерево; `worktree` до старта развёрнут с `node_modules`.

Честность про дыры.

Матрица не притворяется полным забором: часть правил без хука по осознанному решению (параллельные мутации вне `worktree`, обязательная обёртка тестов в `timeout`); под асинхронными `dispatch` часть `post`-хуков слепа; хуки ловят ошибки по привычке, а не намеренное уклонение — это названный `residual`, а не скрытая дыра.

Обслуживание.

Хук, добавленный в сессии, не активен в ней же — проверяется в следующей; любое изменение хука само проходит верификацию отдельным агентом; захардкоженный проектный литерал в хуке обязан попасть в чек-лист форка нового проекта.

Зачем вообще матрица.

Без неё накопились три типа дефектов: правило расширилось — хук продолжал проверять старую формулировку; два хука с общей эвристикой правились независимо и разъехались; новое правило появилось без проверки вообще. Матрица делает все три состояния видимыми при каждом чтении.

Астра: пояснения к главе

Хук — обработчик события в среде выполнения агента. `Shell`-хук выполняет обычную программу; его решение зависит от входных данных, окружения и собственной корректности. Детерминированность означает воспроизводимое правило вычисления, а не безошибочность и не полноту контроля.

Термин	Простое объяснение
RULE → HOOK → ENFORCEMENT	Какое правило существует; какой обработчик его проверяет; что система делает при нарушении.
Blocking; deny; Soft; advisory	Блокирующая проверка; отказ в действии; предупреждение; рекомендация, не препятствующая выполнению.
n/a; coverage gap; residual	Неприменимо; отсутствие нужного покрытия; риск, сохраняющийся после введённых мер. Это разные состояния.
Bash; shell; <code>sed -i</code> ; <code>redirect</code> ; <code>tee</code>	Командная оболочка и способы обработки или записи файлов. Запись возможна не только инструментом редактирования.
PreToolUse; PostToolUse; <code>async</code>	Событие до инструмента; событие после инструмента; выполнение без ожидания завершения в текущем потоке.

Термин	Простое объяснение
Typecheck; ESLint; node_modules	Проверка типов; анализатор JavaScript/TypeScript; каталог установленных зависимостей проекта.
Хардкод; литерал; fork; helper	Значение, зашитое в код; конкретная константа; ответвление или копия проекта; вспомогательная программа.
Timeout; drift; pipeline-log	Предел времени; незапланированное расхождение состояния; журнал прохождения задачи по пайплайну.

Астра: границы механической защиты

Приведённые автором количества и доли описывают его систему. Исходных 53 хуков и полной матрицы в предоставленном материале нет; их фактическое покрытие здесь не проверялось. Нельзя интерпретировать «40% blocking» как «40% безопасности»: одна незакрытая критическая операция может быть важнее десятков закрытых малозначимых правил.

В матрице я разделила бы «неприменимо» и «нужная проверка не реализована». Иначе существенная дыра выглядит как нормальное исключение. Полезные поля: правило, область действия, событие, решение, тест запрета, тест разрешения, владелец, версия и остаточный риск.

В описании есть процессное противоречие: GAP и бюджет итераций названы жёсткими ограничениями, но встречаются в категории Soft. Его нужно разрешить в реальной конфигурации. Если превышение бюджета лишь вызывает предупреждение, предельное число попыток не обеспечено механически.

По документации Claude Code решение PreToolUse может запретить вызов, а PostToolUse выполняется после него. Поэтому последующая проверка не отменяет уже совершённую запись. Поведение обновления конфигурации, ошибок и асинхронных обработчиков нужно проверять для закреплённой версии среды; фразу о неактивности нового хука в текущей сессии нельзя переносить на все среды как универсальный закон. [1]

Перехват знакомых написаний команд полезен против обычных ошибок, но не равен изоляции прав. Запись бывает через интерпретатор, дочерний процесс, библиотеку или другой инструмент. Надёжнее лишить роль ненужных возможностей на уровне среды, а хуками проверять бизнес-правила и порядок работ. Документация sandboxing отдельно рассматривает файловые и сетевые ограничения. [2]

Астра: пример строки матрицы

Поле	Пример предлагаемого контракта
Правило	R-VERIFY: выпуск только проверенного снимка.
Проверка	Перед интеграцией сравнить снимок кандидата и объект, указанный в вердикте.
Решение	Нет вердикта, он повреждён или устарел — запретить выпуск.
Положительный тест	Для неизменённого кандидата есть подлинный PASS со всеми обязательными проверками.

Поле	Пример предлагаемого контракта
Отрицательный тест	После PASS изменена одна строка продукта: выпуск отклоняется.
Остаточный риск	Сам проверяющий может пропустить смысловой дефект; это требует отдельной оценки его качества.

Астра: этапы реализации

1. Составить короткий реестр критических правил

Начать с полномочий записи, запрета запуска без брифа, актуальности вердикта, бюджета попыток и сохранения работы. Для каждого правила описать нарушение через наблюдаемый результат, а не через формулировку промпта. Результат — небольшой реестр с назначенными владельцами.

2. Поместить контроль вне изменяемого агентом контура

Рабочие роли не должны менять активные политики, счётчики и подтверждения проверки. ROLE-маркер полезен как метка, но идентичность роли должен задавать запускающий процесс. Приёмочный сценарий: воркер не может повысить свои полномочия изменением текста отчёта или файла проекта.

3. Проверять вход и выход каждого обработчика

Для хука нужны проверка схемы события, ограничение времени и однозначный результат. Отдельно проверить отсутствие обработчика, его аварийное завершение и некорректный ответ. Для критического действия неопределённость должна закрывать переход, а не превращаться в молчаливое разрешение.

4. Испытать правила в разрешённых и запрещённых случаях

Взять типичные инструменты записи, абсолютные и относительные пути, вложенный запуск команд, асинхронный возврат и отмену. Цель — проверить границу полномочий. Перечень испытаний не следует считать исчерпывающим доказательством защиты от любого возможного обхода.

5. Сделать финальный гейт отдельной точкой контроля

Проверять интегрированный кандидат, результаты тестов, вердикты и журнал в доверенной среде выпуска. Локальная проверка перед коммитом удобна для ранней обратной связи, но окончательное правило должно охватывать все разрешённые пути выпуска.

6. Связать изменения политики с проверкой матрицы

Изменение правила, хука и его тестов должно иметь общую запись причины. Новый обработчик сначала проверяется на известной разрешённой и запрещённой операции, затем включается в работу. Готовность этапа — подтверждённое срабатывание, а не просто наличие файла на диске.

ГЛАВА 03

3. Фундамент: принципы контроля работы агентов

АВТОРСКИЙ ТЕКСТ · АНТОН

Пайплайн стоит на нескольких базовых принципах контроля, каждый из которых — ответ на конкретный отказ LLM-агента как исполнителя:

1. Инициатор $\neq$ контролёр. Воркер никогда не верифицирует собственный вывод; оркестратор никогда не пишет код. Это четырёхглазый принцип (four-eyes) и разделение обязанностей из аудита/внутреннего контроля — применённые к агентам: самоодобрение запрещено структурно, а не уговорами.
2. Спецификация как эталон. Любой вердикт выносится относительно формального артефакта (бриф, спека, §-ссылки), а не относительно «выглядит нормально». Это контроль относительно стандарта, а не относительно мнения.
3. Двухконтурное управление. Поведение агента задаётся инструкцией (LLM-контур, слабый), а контроль — детерминированными хуками (механический контур, сильный). Вебинарная формулировка: инструкция — это рекомендация, хук — это ограждение.
4. Ограниченная автономия. Никакая петля не бесконечна: бюджет итераций (3), бюджет диспетчей, таймаут на один dispatch. Агент может ошибаться много раз — но система останавливает счёт и передаёт решение человеку, а не крутится дальше.
5. Полнота маршрутизации находок. Каждый обнаруженный дефект попадает в один из двух треков (свой дифф — фикс, чужой код — батч-эскалация). «Третьего ящика» нет — это контроль полноты обратной связи: сигнал не может исчезнуть молча.
6. Внешний контур на границах. Неоднозначность (GAP), edge case, исчерпание бюджета — точки выхода к человеку, а не к догадке агента. Автономия локальна, ответственность глобальна.

Астра: пояснения к главе

Здесь автор описывает внутренний контроль: кто может действовать, кто принимает результат и как не потерять сигнал об ошибке. Эти принципы полезны даже в системе с одним исполнителем и одним проверяющим. Количество агентов вторично относительно проверяемых полномочий и ясного эталона.

Термин	Простое объяснение
Four-eyes; разделение обязанностей	Значимый результат подтверждает другой участник, у которого нет права просто одобрить собственную работу.
Артефакт; §-ссылка	Сохранённый объект работы; указание на конкретный раздел требования. Артефакт может быть текстом, диффом или протоколом проверки.

Термин	Простое объяснение
Гейт; GAP; edge case; стаб	Условие допуска; пробел в требованиях; пограничная ситуация; заглушка вместо полноценного поведения.
Бюджет; внешний контур	Предельные ресурсы задачи; человек или другой уровень управления, принимающий решения за пределами полномочий исполнителя.
Маршрутизация; батч-эскалация	Назначение находке дальнейшего пути; передача нескольких находок ответственному одним пакетом.

Астра: независимость и ответственность

У двух агентов может оказаться общая ошибочная предпосылка: одинаковая модель, неверная спецификация или один и тот же неполный бриф. Независимость ролей — организационное свойство; независимость ошибок — эмпирический вопрос. Её стоит оценивать по тому, какие реальные дефекты проходит каждая комбинация ролей.

Требования тоже нуждаются в проверке. Если в исходной спеке для повторного платёжного события случайно разрешено повторное начисление, буквальный спек-верификатор может принять опасное поведение. Поэтому я добавила бы список доменных инвариантов, согласованный владельцем системы, и отдельную проверку противоречий между ним и задачами.

Слово «эскалировано» должно означать не только «записано в файл», но и «назначен ответственный, известен статус». Нужны состояния новой находки, подтверждения, назначения, исправления и закрытия либо явно принятого остаточного риска. Иначе сигнал формально маршрутизирован, но практически потерян.

Старый дефект не всегда можно просто вынести за пределы задачи. Если он делает текущую поставку небезопасной, блокирует проверку или обесценивает результат, работу следует остановить в соответствующей части и передать решение владельцу. Возраст дефекта определяет исполнителя исправления, но не его влияние на выпуск.

Не каждую неопределённость полезно отправлять человеку. Мой вариант: обязательный вопрос для денежных правил, доступа, необратимых эффектов и противоречий критериев; обратимые технические детали внутри согласованных границ выбираются исполнителем и записываются. Такой порог нужно утвердить заранее, чтобы не подменять авторский GAP-гейт молчаливыми догадками.

Астра: этапы реализации

1. Описать защищаемые результаты

Выделить несколько конкретных инвариантов продукта: например, повтор события не создаёт новую операцию; роль пользователя не расширяется из входного запроса; принятый результат соответствует заданной версии требований. Этап завершён, когда для каждого инварианта можно назвать нарушающий пример.

2. Развести создание и утверждение

Назначить отдельные идентификаторы автора и принимающего. Разрешить исполнителю диагностические проверки, но запретить самостоятельный выпуск. Зафиксировать, кто принимает изменения самой спецификации и кто разрешает исключения из процесса.

3. Ввести реестр находок

Минимальная запись: finding_id, задача, серьёзность, доказательство, затронутая версия, владелец, статус и связь с исправлением. Удаление находки без закрывающей причины не допускается. Проверить, что находка в старом коде получает владельца и сохраняет связь с текущей задачей.

4. Настроить порог эскалации

Определить, что требует решения человека и какие альтернативы нужно ему предъявлять. Хорошая эскалация содержит проблему, влияние, уже собранные доказательства и варианты продолжения. Она не должна заставлять человека заново проводить весь анализ.

5. Испытать полноту обратной связи

Подложить учебные случаи: неверный бриф, спорный FAIL, дефект вне диффа и превышение бюджета. По каждому проверить конечный статус. Результат этапа — отсутствие находок без решения или ответственного, а не только отсутствие падений программы.

ГЛАВА 04

4. Перекличка с теорией надёжности

АВТОРСКИЙ ТЕКСТ · АНТОН

- Recovery blocks (Randell, 1975): верификатор — это acceptance test; FAIL запускает alternate (rework-воркер) с сохранением основного состояния. Бюджет из 3 итераций — ограничение на число попыток recovery.
- N-version programming (Avizienis): параллельная пара «тест-агент вслепую к коду / код-агент вслепую к тестам» из одного брифа — проектная диверсификация каналов. Общий контракт имён из брифа играет роль общего спецификационного интерфейса — без него версии расходятся в деталях, что и есть классическая проблема NVP.
- Цепочка fault → error → failure (Avizienis-Laprie): дефект ловится на стадии fault (верификатором до коммита), до того как станет error'ом в дереве и failure'ом у пользователя. Эшелоны: инструкция → хук → верификатор → коммитные гейты — это defence in depth: независимые рубежи, каждый со своим классом пропусков.
- Разнообразие против common-cause failures: одинаковые агенты слепы одинаково. Разделение моделей по роли (сильная — контроль, дешёвая — генерация) и разных информационных каналов (тесты из брифа, верификация по исходной спеке) снижает корреляцию отказов. И обратный урок из той же теории: дефект может маскироваться под шум (симптомы гонок за общую БД выглядят как «флаки инфраструктуры») — приписывать шуму запрещено, пока есть конкурирующие исполнители.
- Watchdog-таймер: таймаут dispatch — классический сторожевой таймер; зависший исполнитель принудительно переводится в терминальное состояние, и это выглядит как «нормальное завершение» — поэтому событие смерти синтезируется и доставляется отдельно.
- Interlock и fail-closed: хуки — программные блокировки; в неоднозначных случаях они запрещают (deny), а не разрешают — fail-safe направление по умолчанию.
- Checkpointing: перед любым откатом состояние строки сохраняется как артефакт — тот же мотив, что и контрольные точки/wal-журналирование; запись pipeline-log обязана ехать в одном коммите с кодом (log first, как в WAL).
- Graceful degradation: блокировка части шага не останавливает сессию — заблокированная часть паркуется, остальное продолжается (деградация сервиса вместо отказа).
- V-модель: тесты выводятся из спецификации независимо от реализации — правая ветвь V как отдельный агент, а не пересказ кода.
- PDCA (Shewhart-Deming): сам основной цикл — plan (бриф) → do (воркеры) → check (верификаторы) → act (фикс/эскалация), с ограничением числа витков.

Астра: пояснения к главе

Теория надёжности даёт полезный язык для анализа пайплайна. Однако сходство с классическим методом не переносит автоматически его гарантии. В этой главе особенно важно различать инженерную аналогию, реализованный механизм и доказанное свойство.

Термин	Простое объяснение
Recovery block; acceptance test; alternate	Попытка выполнить работу с проверкой приемлемости; сама проверка; альтернативный вариант после отказа.
N-version programming; NVP	Несколько независимо разработанных реализаций одной функции с механизмом сопоставления или выбора результатов.
Fault; error; failure	Причина дефекта; ошибочное внутреннее состояние; наблюдаемое нарушение требуемого сервиса.
Common-cause failure; корреляция	Общая причина нескольких отказов; зависимость ошибок участников друг от друга.
Watchdog; interlock; fail-closed	Сторожевой таймер; блокировка недопустимого действия; отказ в доступе или переходе при неопределённости.
Checkpoint; WAL; graceful degradation	Сохранённая точка состояния; журнал предзаписи; продолжение доступной части работы при отказе другой части.
V-модель; PDCA	Связь уровней требований с уровнями проверки; цикл «планировать → делать → проверять → улучшать».
Гонка; flaky-тест	Зависимость результата от порядка одновременных действий; тест, который нестабильно проходит при внешне одинаковых условиях.

Астра: где аналогии требуют оговорок

Пара «тесты + реализация» не является буквальным N-version programming: участники создают разные виды артефактов, а не две реализации одной функции. Ближе формулировка «независимое выведение проверок и реализации из требований». Recovery blocks тоже служат аналогией: здесь дорабатывается программа, тогда как классическая схема описывает восстановление выполнения с альтернативами. [9]

Fault, error и failure не соответствуют этапам «до коммита», «в дереве» и «у пользователя». Ошибка в формуле — fault; неверная сумма в состоянии выполняющейся программы — error; выдача неправильного результата через интерфейс — failure. Такой отказ может наблюдаться уже в тестовой среде. Различаются понятия и граница наблюдения, а не стадии Git-процесса. [9]

Сохранение журнала вместе с кодом улучшает прослеживаемость, но ещё не создаёт WAL. Предзапись означает сохранение записи до соответствующего изменения постоянных данных с возможностью восстановления после сбоя. Это свойство порядка и устойчивости записи; обычный совместный коммит двух файлов его не доказывает. [10]

Разные модели и свежие контексты могут давать разные находки, но степень снижения общей ошибки нужно измерять. Несколько рубежей не следует называть независимыми без проверки: общий неверный контракт способен пройти через тесты, реализацию и спек-верификатор одновременно.

Fail-closed полезен на переходе к опасной операции, но не означает, что остановка всей системы всегда безопасна. Для каждого действия надо определить безопасное состояние. При техническом сбое можно запретить выпуск новой версии, сохранив работающую предыдущую. Таймаут исполнителя должен иметь собственный статус, а не маскироваться под успешное завершение.

Астра: учебный пример общей причины отказа

Предположим, из исходного требования «повтор события не меняет баланс» брифер потерял слово «не». Тест-агент закрепил повторное начисление, код-агент его реализовал, а проверяющий прочитал только бриф. Три роли согласны, но требование нарушено. Дополнительный независимый путь — сверка с первичным требованием и отдельно утверждённым инвариантом — обнаруживает ошибку.

Астра: этапы реализации

1. Составить каталог конкретных отказов

Для каждого риска описать причину, ошибочное состояние и наблюдаемый вред. Включить неверный бриф, подложный PASS, потерянный возврат агента, гонку тестов и сбой сохранения. Этап завершён, когда у каждой записи есть проверяемый сценарий.

2. Назначить рубеж обнаружения и восстановления

Указать, что обнаруживает хук, что подтверждает тест, что оценивает верификатор и что делает управляющий процесс после отказа. Проверка без определённого действия по её результату остаётся только наблюдением.

3. Испытать контрольные точки

Сохранить не только дифф, но и исходную версию, незакоммиченные новые файлы, состояние задачи и ссылки на отчёты. В учебной копии прервать процесс и восстановить его. Успех — возврат к понятному состоянию без потери работы и без повторения уже совершённого внешнего эффекта.

4. Провести испытания с намеренно внесёнными сбоями

Остановить воркер, задержать ответ, повредить отчёт, дважды доставить завершение. Проверить конечный статус и дальнейшие полномочия. Это fault injection: контролируемое создание отказов для проверки самой системы управления.

5. Измерить полезность каждого рубежа

Сравнить, какие дефекты ловят тесты, разные верификаторы и их сочетание. Записывать пересечения пропусков. Добавлять очередного проверяющего имеет смысл, когда он обнаруживает значимые дополнительные дефекты при приемлемой стоимости.

ГЛАВА 05

5. Пайплайн как конечный автомат

АВТОРСКИЙ ТЕКСТ · АНТОН

Уровень 1 — цикл воркер/верификатор:

CLAIM → BRIEF → DISPATCH_WORKER → VERIFY

VERIFY —PASS—→ COLLECT → COMMIT → CLAIM (следующая строка)

VERIFY —FAIL—→ REWORK(iter+1) → VERIFY

guard: iter < 3; при iter = 3 → ESCALATE (терминальная, дифф сохранён)

Переходы охраняются: DISPATCH требует непустого брифа (иначе нет диспетча), COMMIT — записанного вердикта и pipeline-log записи, REWORK — бюджета итераций.

Уровень 2 — сам агент.

Каждое действие агента — переход, и на каждом ребре стоит guard: PreToolUse-хук — предикат перед переходом; deny запрещает переход (автомат остаётся на месте и получает причину — обязан выбрать другое ребро); PostToolUse-хук — пост-условие/наблюдатель, который не блокирует, но оставляет след. Это в точности guarded commands в духе Дейкстры: ребро недоступно, пока охранник ложен.

Уровень 3 — время.

Ожидание вне переходов запрещено (sleep запрещён): автомат событийный, переходы происходят по возврату агента, а не по таймеру. Единственный легальный таймер — watchdog dispatch (60 мин), то есть ограничение на пребывание в состоянии — фактически clock constraint из timed automata.

Уровень 4 — параллелизм.

Волна — это N независимых копий лэйн-автомата (brief → worker → verify → merge → teardown), каждый в своём worktree. Общий ресурс (главное дерево) защищён мьютексом на merge; закрытие волны — барьерная синхронизация: все лэйны достигли терминального состояния, один закрывающий коммит.

Уровень 5 — иерархия.

Доска строк — множество параллельных автоматов с инвариантами (чистое дерево, чистый write-set). Автомат агента вложен в автомат цикла, цикл вложен в план-цикл владельца — иерархический автомат (statechart по Харелу): ошибка не разрешается на своём уровне → эскалируется вверх как внешнее событие для верхнего автомата. Ждущие состояния (PLAN-BLOCKED, ожидание решения владельца) — состояния, из которых выход только внешним входом; сессия на них не блокируется, а переключается на другие строки.

Практический смысл всей рамки: переходы наблюдаемы и гейтуемы, состояние пишется в артефакты (доска, вердикты, pipeline-log) — машина воспроизводима и аудируема.

LLM-инструкция управляет выбором переходов; хуки — допустимостью переходов; верификатор — валидностью результата перехода.

Астра: пояснения к главе

Конечный автомат — описание допустимых состояний и переходов. Он полезен тем, что слово «почти готово» заменяется конкретным состоянием: например, «реализация завершена, независимая проверка ещё не получена». Состояние хранит управляющая программа; рассказ агента о нём служит входным сообщением, которое ещё надо проверить.

Термин	Простое объяснение
State; transition; guard; invariant	Состояние; переход; условие перехода; правило, которое должно сохраняться во всех допустимых состояниях.
CLAIM; COLLECT; ESCALATE	Взять задачу в работу; собрать результаты; передать решение на следующий уровень.
Lane; wave; barrier; teardown	Отдельная цепочка работы; группа одновременно запущенных цепочек; ожидание их завершения; освобождение временных ресурсов.
Write-set; statechart	Набор изменяемых объектов; иерархическое описание состояний, включая вложенные и параллельные процессы.
Timed automata; clock constraint	Автоматы с учётом времени; ограничение времени пребывания или перехода.
Lease; fencing token	Ограниченное по сроку владение задачей; номер поколения полномочий, по которому отклоняют запоздалые действия старого владельца.

Астра: уточнения модели состояний

Назвать процесс автоматом недостаточно для воспроизводимости. Нужны версии входов, журнал событий, сохранённое состояние, определённая обработка повторов и однозначные правила возобновления. Можно воспроизвести последовательность принятых решений, не обещая, что новый запрос к LLM выдаст тот же текст.

Три итерации нужно выразить одним счётчиком. Если первая попытка имеет номер 1, то FAIL после попыток 1 и 2 допускает доработку, а после 3 — эскалацию. Если счётчик обозначает только повторные исправления, общее число запусков будет другим. Это нельзя оставлять на интерпретацию агента.

VERIFY должен уметь завершаться не только PASS или FAIL. Отсутствие среды, таймаут и повреждённый отчёт не доказывают дефект кода. Я предлагаю отдельные статусы INCONCLUSIVE, ERROR, CANCELLED и TIMEOUT. До выяснения причин они не разрешают выпуск, но учитываются отдельно при оценке качества исполнителя.

Git worktree создаёт дополнительные рабочие деревья в рамках репозитория; это не отдельная машина и не отдельная база данных. [3] Поэтому лэйны должны разделять также тестовые БД, порты, временные каталоги, очереди и внешние ресурсы. Иначе текстовые изменения не конфликтуют, а выполнение тестов мешает соседу.

Последовательный merge не заменяет интеграционную проверку. Модуль А и модуль В могут проходить тесты по отдельности, но нарушать общий контракт вместе. Блокировка должна охватывать создание итогового кандидата и проверку его состояния, а выпуск — ссылаться именно на него.

Запрет sleep — локальная политика экономии и управления агентом, а не необходимое свойство событийной архитектуры. Управляющая программа может использовать таймеры повторного подключения и ограниченный backoff — увеличение задержки между попытками. Такие ожидания должны быть наблюдаемыми, ограниченными и отменяемыми.

Астра: пример таблицы переходов

Состояние и событие	Условие	Новое состояние
BRIEF_READY → запуск	Полный бриф, свободный исполнитель, попытка зарезервирована	RUNNING
RUNNING → результат	Верный task_id, активная попытка, снимок сохранён	VERIFYING
VERIFYING → PASS	Все обязательные проверки актуальны для кандидата	ACCEPTED
VERIFYING → FAIL	Есть содержательный дефект; попытка меньше 3	REWORK_READY
VERIFYING → FAIL	Использованы три попытки	ESCALATED
Любое активное → таймаут	Истёк срок; полномочия отозваны	TIMED_OUT
ACCEPTED → интеграция	Итоговый кандидат проверен, общая блокировка получена	COMMITTED

Астра: этапы реализации

1. Описать состояния и запрещённые переходы

Сначала составить таблицу, затем реализовать обработчик переходов. Для каждого события указать обязательные поля, условия и результат отказа. Приёмочный пример: событие «коммитить» из RUNNING отвергается независимо от формулировки сообщения агента.

2. Сохранять изменения состояния атомарно

Резервирование попытки и запись её запуска должны образовывать одну согласованную операцию. Для небольшой установки подойдёт транзакционное локальное хранилище; для нескольких процессов — хранилище с контролем конкуренции. Файлы с человеческими отчётами можно формировать из состояния, а не использовать как единственный источник истины.

3. Обработать повторы и запоздалые сообщения

Каждое событие получает уникальный идентификатор. Повторная доставка не создаёт повторный коммит. Возврат после таймаута сохраняется для анализа, но не переводит задачу в PASS автоматически: идентификатор попытки и поколение полномочий уже не актуальны.

4. Вынести watchdog в управляющий процесс

Таймер должен работать независимо от того, вызывает ли зависший агент инструменты. При истечении срока отменяются исполнитель и его дочерние операции в пределах предоставленных полномочий. Статус сохраняется, а очередь получает событие, по которому возможна доработка или эскалация.

5. Добавить очередь интеграции и барьер волны

Кандидаты объединяются по одному. При конфликте создаётся отдельная задача исполнителю; координатор не правит продукт сам. После каждого существенного изменения интегрированного кандидата необходимые проверки повторяются. Волна закрывается, когда у всех участников есть терминальный статус и сохранённый результат.

6. Испытать возобновление с середины

Перезапустить управляющий процесс во время проверки и интеграции. Готовность означает, что система различает «ещё не сделано», «сделано» и «результат неизвестен», не теряет блокировки и не повторяет значимое действие без выяснения его предыдущего результата.

ГЛАВА 06

6. Пайплайн против рынка: объективная сверка

АВТОРСКИЙ ТЕКСТ · АНТОН

Опорные точки рынка (2025–2026): Anthropic «How we built our multi-agent research system» и официальные best practices Claude Code; OpenAI Agents SDK; CrewAI; Cognition «Don't Build Multi-Agents».

Что у нас совпадает с рынком

Каркас пайплайна — мейнстрим, ничего экзотического. Оркестратор, который сам не пишет код, а раздаёт задачи и собирает результаты, — это *orchestrator-worker* у Anthropic и *hierarchical process* у CrewAI, где менеджер «планирует, делегирует, валидирует». Бриф с чёткими границами — тот же вывод, к которому пришла Anthropic после инцидентов: «расплывчатые инструкции приводили к дублированию работы», только у нас он формализован как обязательный артефакт.

Верификатор отдельным агентом в свежем контексте — официальная рекомендация Claude Code («add an adversarial review step»): ревьюер видит только дифф и критерии, но не рассуждения автора. Наш адверсальный верификатор — тот же паттерн, усиленный обязательством предъявить конкретный атакующий сценарий, а не перечитать спеку.

Спаренный кодген тоже рыночный: docs Claude Code прямо советуют «пусть один Claude пишет тесты, другой — код, который их проходит». Наше отличие — это рекомендация у рынка, а у нас обязательная фаза с запретом агентам читать вывод друг друга.

Параллелизм через *git-worktree* — рынок: официальный механизм *worktree*-сессий и *fan-out* на 5–30 сабагентов, каждый в своём *worktree* и со своим PR. Бюджеты итераций и эскалация к человеку на границах — есть во всех фреймворках; у Anthropic это масштабирование усилия в промпте, у нас — счётчики.

Где наш пайплайн жёстче рынка

Детерминированный слой контроля. Рынок знает хуки как точечные гейты («действие должно происходить всегда, без исключений»), но типовой пайплайн опирается на промпты и верификаторов. У нас — матрица *RULE→HOOK*, где «правило без хука» — публично зафиксированный пробел, а не молчаливый пропуск. Дисциплины контроля самих правил (анти-дрейфт текста правил и хуков, аудит покрытия) в рыночных фреймворках нет.

Протокол верификации. У рынка верификатор обычно возвращает текст; у нас — контракт: вердикт, фидбек-файл для *gework*-воркера, учёт итераций, разделение находок по классам (косметика не может одна сделать FAIL). Ближайший аналог — рубричный *LLM-as-judge* Anthropic.

Двухтрековая маршрутизация дефектов (свой дифф — правится немедленно; чужой/старый код — батч-эскалация владельцу, «третьего ящика» нет) — на рынке в таком виде не стандартизирована.

Сохранение работы перед откатом, парковка блокеров — ручные аналоги checkpointing; в фреймворках это делегируется платформе, а не протоколу.

Где рынок объективно впереди

Первое и главное — evals и калибровка верификатора. Anthropic ведёт статистику: рубричный судья с оценкой 0–1, небольшая выборка реальных запросов с первого дня, оценка конечного состояния для мутирующих агентов. У нас верификатор не измеряется: нет метрик пропусков и ложных FAIL, нет регрессионного тестирования самого контроля. Это систематический пробел.

Дальше — трейсинг (у OpenAI SDK и LangSmith решения инструментированы; у нас текстовый журнал), sandboxing (OpenAI и Claude Code изолируют сеть и файлы на уровне ОС, у нас изоляция только git-уровня), fail-fast guardrails (у OpenAI проверки бегут параллельно с агентом и останавливают его до деструкции — у нас то же по смыслу, но у них первоклассный API), и стоимость/скорость — рынок оптимизирует их, наша церемониальность платит латентностью.

Самая спорная наша ставка

Cognition прямо против параллельных агентов: конфликтующие неявные решения дают плохие результаты, рекомендация — однопоточный агент со сжатием контекста. Наш дизайн — противоположный полюс: мутации параллельно в worktree. В нашу пользу — mitigation, которого у Cognition не рассмотрено (изоляция деревьев, бриф-контракт с предопределёнными именами, reconciliation, верификатор), и то, что вендор нашего же харнесса параллельные worktree-сессии официально рекомендует. Но этот спор рынок не закрыл — в вебинаре его стоит называть самым рискованным архитектурным решением, а не фактом.

Вердикт

Архитектура внутри мейнстрима; отличие — в плотности контроля: детерминированный слой и протоколы верификации развиты дальше рыночных, статистическая проверка самого контроля (evals) — слабее. Для соло-дева и продукта с деньгами на борту расклад оправдан; для команды в быстрой разработке та же церемониальность была бы перебором.

Астра: пояснения к главе

Термин	Простое объяснение
Framework; SDK; harness	Каркас для сборки приложений; набор средств разработки; среда, которая запускает агента и управляет его инструментами и жизненным циклом.
Evals; LLM-as-judge; калибровка	Набор измерительных задач; модель в роли оценщика; сравнение её оценок с надёжной человеческой или программной разметкой.
Tracing; span; latency	Связанный след исполнения; отдельная операция внутри него; задержка от начала работы до результата.

Термин	Простое объяснение
Sandbox; guardrails; fail-fast	Ограниченная среда выполнения; защитные проверки; раннее прекращение заведомо недопустимой работы.
Fan-out; PR; reconciliation	Раздача работы нескольким исполнителям; предложение изменения в репозитории; согласование и устранение расхождений.
Mitigation; baseline; рубрика	Мера снижения риска; базовый вариант для сравнения; набор критериев оценивания.

Астра: сверка и обновления на 12 сентября 2026 года

Общая схема «менеджер → исполнители» действительно присутствует у Anthropic и в иерархическом процессе CrewAI. Это подтверждает распространённость архитектурного паттерна, но не означает, что каждый такой менеджер технически лишён права писать код. Этот запрет — дополнительное свойство конкретной реализации. [4, 5]

Документация Claude Code описывает отдельные контексты автора и ревьюера, передачу тестов другому исполнителю и распределение некоторых массовых задач на 5–30 worktree-сабагентов. Важная деталь: последовательность «один пишет тесты, другой проходит их» сама по себе не означает одновременную разработку вслепую. [6]

22 апреля 2026 года Cognition опубликовала «Multi-Agents: What’s Actually Working». Компания описывает работающий цикл кода и ревью с чистым контекстом, а также другие формы взаимодействия агентов. Её предпочтение по-прежнему — последовательные записи при участии нескольких агентов в анализе. Поэтому актуальный спор касается прежде всего одновременных независимых авторов изменений, а не любого использования нескольких агентов. [7]

Утверждение о guardrails, которые при параллельном запуске обязательно остановят агента «до деструкции», слишком сильное. Документация OpenAI Agents SDK прямо предупреждает: до завершения параллельной входной проверки агент может успеть вызвать инструменты. Для предотвращения таких эффектов предусмотрено блокирующее выполнение; охват проверок зависит также от типа инструмента. [8]

Фразы «есть во всех фреймворках», «у рынка обычно» и «развиты дальше рыночных» нельзя подтвердить несколькими примерами документации. Это авторские оценки. Для объективного превосходства нужна одинаковая выборка задач, сопоставимые ограничения и измеренные результаты. Также наличие sandboxing в продукте ещё не доказывает, что все операции конкретного запуска действительно изолированы.

По evals позиция автора остаётся существенной: Anthropic рекомендует сочетать программные, модельные и человеческие оценки, проверять конечный результат и калибровать судей. [11] Мой вывод: измерять нужно не только, насколько часто верификатор говорит PASS, но и насколько часто он принимает скрыто неправильный результат.

Астра: этапы реализации

1. Задать честную базу сравнения

Взять два режима: один исполнитель с независимым ревью и полный авторский пайплайн. Использовать одинаковые исходные версии, задачи, критерии и ограничения. Отдельно учитывать расход модели и человеческое время.

2. Собрать репрезентативные задачи

Включить новый модуль, багфикс, изменение интерфейса, интеграцию двух модулей, чувствительную операцию и задачу с неполной спецификацией. Не выбирать только хорошо распараллеливаемые примеры. Начальная выборка — инструмент поиска крупных проблем, а не доказательство низкой частоты редких отказов.

3. Закрепить версии сравниваемых конфигураций

Сохранить модели, параметры, промпты, политики, версию среды, зависимости и бюджет. Возможности фреймворка записывать вместе с конкретной конфигурацией, которая их включает. Обновление модели или политики считается новой конфигурацией эксперимента.

4. Измерить результат задачи целиком

Собирать успешность независимой приёмки, пропущенные дефекты, ложные отказы, число доработок, время до принятия, расходы и минуты участия человека. Не подменять эти метрики скоростью выдачи токенов или количеством завершённых диспетчей.

5. Выбирать режим по измеренному классу задач

Оставить полный процесс там, где он снижает значимые пропуски или полную стоимость исправления. Упростить низкорисковые задачи, если дополнительные роли только удлиняют очередь. Повторять сравнение после существенных изменений моделей и правил, не переносить выводы бессрочно.

ГЛАВА 07

7. Разбор Астры: от дисциплины агентов к проверяемой системе

Автор этой главы, пояснений и планов внедрения — Астра, ИИ-ассистент. Ниже изложен инженерный анализ предоставленной архитектуры и проверенных первоисточников. Это не отчёт об эксплуатации исходного репозитория: его код, хуки и статистика запусков не предоставлялись. Учебные примеры и предложенные контракты не являются результатами выполненных испытаний.

7.1. Что в подходе особенно ценно

Главная сильная сторона — явный маршрут от задачи до принятия результата. Если бриф, права, вердикт и переходы действительно проверяются программой, система меньше зависит от того, насколько убедительно агент описывает собственный успех. Матрица покрытия делает видимыми места, где техническое ограничение пока существует только на словах.

Второй полезный элемент — отдельный контекст проверки. Он позволяет заново задать вопросы к решению, не наследуя всю цепочку оправданий автора. Однако независимость нужно поддержать первичными требованиями, доказательствами и правами среды. Ещё один агент без этих условий может стать дополнительным источником уверенного, но неверного одобрения.

Третий — ограниченная доработка. Предел попыток превращает неопределённый расход ресурсов в управляемый процесс. Он работает, если сохраняется между перезапусками, считается доверенным компонентом и не сбрасывается переименованием задачи.

7.2. Контроль должен принимать доказательства

Я бы сделала центральным объектом не текст «готово», а пакет доказательств для конкретного кандидата. Он связывает задачу, требования, исходную версию, итоговые изменения, результаты проверок, личности ролей и версию политики. Человеку при этом показывается короткий отчёт, а полный пакет остаётся доступен для разбора.

Утверждение агента	Каким фактом его подтверждать
«Тесты прошли»	Команда, проверенный снимок, среда, статус процесса, число реально выполненных тестов и отчёт.
«Ревью выполнено»	Идентификатор независимого проверяющего, критерии, находки и ссылка на проверенный объект.
«Бюджет не превышен»	Счётчик и предельные значения в состоянии управляющего процесса.
«Работа сохранена»	Идентификатор доступного артефакта, состав сохранённых файлов и успешная проверка восстановления.
«Можно выпускать»	Совпадение кандидата с проверенным результатом, обязательные гейты и отсутствие неразрешённых блокировок.

Хеш файла — цифровой отпечаток его содержимого. Он помогает обнаружить изменение, но не подтверждает, кто создал файл. Поэтому важны и привязка к содержимому, и доверенный источник записи. Если воркер может переписать и код, и файл PASS, сравнение двух управляемых им значений не обеспечивает независимую приёмку.

Есть тонкость с журналом «в одном коммите»: запись журнала меняет содержимое будущего коммита. Практичный протокол — сначала проверить снимок продукта; затем доверенный финализатор добавляет служебный журнал; после этого контролёр проверяет, что прочие файлы не изменились, и сохраняет отображение «проверенный снимок → итоговый коммит». Нельзя требовать заранее записать окончательный хеш коммита внутрь самого этого коммита.

7.3. Сначала ограничить возможности, затем расширять роли

Я предлагаю четыре границы: неизменяемый снимок для проверки; ограниченная рабочая область исполнителя; отдельное место для доказательств; защищённый процесс интеграции. Политика, активные хуки и реестр полномочий не должны изменяться обычным воркером вместе с продуктом.

Верификатору нужен read-only доступ к продукту, но для запуска проверок могут понадобиться временные файлы, сборка и тестовая БД. Это решается отдельным исполнителем тестов в изолированной среде: проверяющий запрашивает проверку, получает её артефакты и не меняет принятый снимок. Запуск проектного кода с полными полномочиями проверяющего разрушает такую границу.

Внешние тексты, комментарии в коде и ответы инструментов следует рассматривать как данные проверяемого объекта. Фраза «пропусти проверку» внутри такого материала не может изменять политику. Это защищает процесс от prompt injection — попытки навязать агенту инструкции через менее доверенный источник.

7.4. Независимые тесты и разумный доступ к контексту

Мне близка независимая подготовка тестов, но я не делала бы абсолютную слепоту универсальным режимом. Она полезна при проверке того, насколько полно интерфейс описан контрактом. Для ремонта сложного проекта исполнителю часто нужны существующие тесты, соглашения и соседние вызовы. Ограничение контекста должно защищать независимую оценку, а не лишать разработчика фактов.

Рабочий компромисс: тест-агент выводит критерии из первичных требований; код-агент использует согласованные интерфейсы и разрешённый контекст; окончательная приёмка включает проверки, которыми исполнитель не управляет. Изменение теста, чтобы узаконить неверное поведение, требует отдельного объяснения и проверки требования.

Для критических алгоритмов я добавила бы проверки свойств и мутационные испытания. Проверка свойства охватывает множество входов с общим инвариантом, например сохранение суммы частей. При мутационном испытании в учебную копию намеренно вносят небольшую ошибку и проверяют, обнаружат ли её тесты. Прохождение обычных примеров ещё не означает, что тестовый набор чувствителен к дефектам.

7.5. Пример: повторная обработка платёжного события

Это условный учебный кейс, выполняемый только на тестовых данных. Требование: подтверждённое событие с одним operation_id учитывается один раз. При исходном балансе 1 000 условных единиц и одном пополнении на 200 результат — 1 200, сколько бы раз ни доставлялось то же событие. Конкретный технический механизм зависит от продукта.

Этап	Действие	Доказательство завершения
Бриф	Уточнить идентичность события, валюту, точность сумм, доверие к источнику и конкуренцию.	Контракт и критерии без неразрешённых денежных правил.
Тест-агент	Подготовить случаи одного события, повторов, одновременных повторов и разных событий.	Ожидаемые состояния и тесты из требования.
Код-агент	Реализовать атомарное применение операции в пределах контракта.	Изменение продукта в разрешённых файлах.
Спек-проверка	Сопоставить каждый критерий с кодом и результатами выполнения.	Покрытие критериев и актуальный вердикт.
Адверсальная проверка	Испытать конкурентную доставку и сбой между сохранением разных частей операции.	Фактическое состояние тестовой БД и журнал операций.
Интеграция	Повторить значимые проверки на объединённом кандидате.	Один принятый результат с известной версией.

Если тестовая среда не позволяет воспроизвести конкуренцию, честный результат — «сценарий не проверен», а не PASS на основании чтения кода. Идемпотентность означает, что повтор одной логической операции не создаёт дополнительного эффекта. Для возобновляемых процессов это должно быть свойством самой операции или её доверенной обёртки, а не пожеланием в промпте.

7.6. Как измерять верификатора и стоимость

Нужны размеченные корректные и дефектные результаты. На них отдельно считаются доля пропущенных дефектов и доля ложных отказов. Таймауты и неуверительные вердикты учитываются отдельной строкой. Иначе инфраструктурная ошибка загрязняет оценку рассуждения, а «не смог проверить» случайно выглядит как правильный отказ.

Антропик рекомендует начинать evals с реальных отказов, сочетать разные способы оценивания и проверять модельных судей по человеческой разметке. [11] Моя практическая адаптация — держать часть примеров отдельно от настройки промптов, повторять нестабильные случаи и сохранять исходные отчёты для объяснения каждой оценки.

Метрика	Как считать в этом процессе
Пропуск дефекта	Дефектные кандидаты с PASS / все известные дефектные кандидаты.
Ложный отказ	Корректные кандидаты с содержательным FAIL / все известные корректные кандидаты.
Неполная проверка	TIMEOUT, ERROR, INCONCLUSIVE / все запуски проверки.
Стоимость принятой задачи	Все расходы серии, включая неудачи и доработки / число независимо принятых задач.
Время до принятия	От постановки до принятого кандидата, включая ожидание, ревью и интеграцию.

Метрика	Как считать в этом процессе
Человеческое участие	Минуты постановки, уточнений, разбора отказов и ручной приёмки.

Численный пример расчёта, не результат эксперимента: 80 дефектных кандидатов, из них 8 получили PASS — пропуск 10%. Из 20 корректных кандидатов 3 получили FAIL — ложный отказ 15%. Если все случаи взяты из одной простой функции, эти цифры не описывают надёжность всей системы.

Дешёвая модель выгодна, когда дешевле принятая задача при сопоставимом качестве. Снижение цены токена может перекрываться дополнительными попытками, ревью и временем человека. Выбор модели по роли полезно автоматизировать через конфигурацию, но саму конфигурацию нужно пересматривать по измерениям, а не по ярлыкам «сильная» и «дешёвая».

7.7. Что я изменила бы в приоритетах

Сначала — запрет выпуска непроверенного или изменившегося кандидата, защищённые полномочия и устойчивое состояние. Затем — качество брифа и верификатора. После этого — наблюдаемость и восстановление. Массовый параллелизм я добавляла бы последним: он усиливает и полезную производительность, и нерешённые ошибки координации.

Часть инфраструктуры сохранения состояния можно взять готовой. Например, LangGraph различает контрольные точки состояния конкретного потока и долговременное хранилище; сохранение в памяти процесса не переживает перезапуск. [12] Но наличие checkpoint не делает внешнее действие автоматически однократным: это отдельное требование к операции.

Мне также кажется полезным добавить статус спорной находки с арбитражем. Верификатор способен ошибиться, а исполнитель может предъявить опровергающий пример. Решение принимает независимый арбитр или человек по тем же требованиям; простое несогласие автора не отменяет FAIL. Бюджет арбитража должен входить в общий бюджет задачи.

Астра: дорожная карта внедрения

Этап 1. Минимальный управляемый цикл

Сделать один последовательный маршрут: бриф → исполнитель → независимая проверка → принятие или эскалация. Сохранять версии и три попытки в управляющем процессе. Условие перехода дальше — успешные учебные случаи PASS, FAIL и исчерпание бюджета.

Этап 2. Защищённая приёмка

Ограничить запись ролей, защитить политики, привязать вердикт к кандидату и добавить доверенную интеграцию. Условие готовности — невозможность выпустить изменённый после ревью код или принять поддельный отчёт обычным разрешённым путём.

Этап 3. Измерение качества

Собрать размеченные дефекты и корректные изменения, измерить пропуски, ложные отказы и полную стоимость. Зафиксировать базовый режим для сравнения. Условие готовности — способность объяснить каждый тип ошибки и увидеть регрессию после изменения конфигурации.

Этап 4. Восстановление и наблюдаемость

Ввести устойчивые события, идентификаторы попыток, watchdog, обработку дублей и сохранение незавершённой работы. Условие готовности — прерывание и возобновление на каждом значимом переходе без потери доказательств и повторного внешнего эффекта.

Этап 5. Ограниченный параллелизм

Сначала разрешить параллельных читателей, затем две независимые цепочки с отдельными тестовыми ресурсами и очередью интеграции. Условие готовности — выигрыш времени до принятия без роста значимых пропусков и стоимости разбора конфликтов.

Этап 6. Постоянное сопровождение контроля

Назначить владельца политик и набора evals, ввести проверку изменений хуков и регулярный разбор пропусков. Обновлять маршрутизацию моделей по результатам задач. Зрелая система отличается тем, что может показать, какие ограничения действительно действуют, что они пропускают и во что обходится принятый результат.

Приложение А. Шаблоны рабочих артефактов

Предлагаемые Астрой поля для первого внедрения. Это примеры контракта, а не готовая конфигурация определённого фреймворка. Обязательные поля и права записи нужно закрепить в собственной среде.

А.1. Бриф задачи

Группа полей	Что записать
Идентичность	task_id; версия брифа; исходный commit; версия спецификации.
Назначение	Цель; ожидаемое поведение; что входит в задачу и что исключено.
Доступ	Разрешённые файлы чтения и записи; допустимые инструменты; рабочая среда.
Контракт	Имена; типы; обязательные поля; коды ошибок; единицы; правила границ.
Приёмка	Критерии с идентификаторами; команды проверки; ожидаемые результаты; обязательные сценарии.
Управление	Попытка; предел времени и расходов; открытые вопросы; маршрут эскалации.

А.2. Вердикт независимой проверки

Группа полей	Что записать
Происхождение	task_id; attempt_id; reviewer_id; роль; время; версия политики.
Объект проверки	Идентификатор снимка; исходная версия; версия брифа и требований.
Доказательства	Критерий → проверка → артефакт → наблюдаемый результат.
Находки	finding_id; серьёзность; воспроизведение; ожидаемое и фактическое поведение; владелец.
Итог	PASS, FAIL либо причина неполной проверки; непройденные обязательные критерии.

А.3. Пакет передачи на доработку

Ссылка на неизменяемый вердикт; список открытых finding_id; актуальная версия брифа; разрешённая область изменений; оставшийся бюджет; способ повторной проверки. Оркестратор передаёт идентификатор пакета, а не переписывает замечания своими словами.

А.4. Контрольные вопросы читателю

1. Что не позволяет исполнителю заменить код после получения PASS?

2. Как система отличает неправильный результат от проверки, которая не состоялась?

3. Где хранится число попыток после перезапуска и смены агента?

4. Какие ресурсы остаются общими у двух worktree и как это повлияет на тесты?

5. Кто получает старый дефект, который делает текущий выпуск неприемлемым?

6. По каким наблюдениям будет понятно, что ещё один агент улучшил результат?

Источники и дальнейшее чтение

Первоисточники, использованные для пояснений и анализа Астры. Проверены 12 сентября 2026 года. Документация продуктов может обновляться; при внедрении фиксируйте версию своей среды.

[1] [Claude Code. Hooks reference](#)

События до и после инструментов, решения обработчиков и ограничения конкретной среды.

[2] [Claude Code. Configure the sandboxed Bash tool](#)

Файловые и сетевые границы среды выполнения.

[3] [Git. git-worktree Documentation](#)

Назначение рабочих деревьев и их связь с общим репозиторием.

[4] [Anthropic. How we built our multi-agent research system](#)

Оркестратор и исполнители; требования к передаваемому контексту.

[5] [CrewAI. Processes](#)

Последовательный и иерархический процессы; роль менеджера.

[6] [Claude Code. Best practices](#)

Отдельный контекст ревьюера, разделение тестов и кода, параллельные worktree-задачи.

[7] [Cognition / Walden Yan. Multi-Agents: What's Actually Working. 22.04.2026](#)

Обновление позиции после статьи Don't Build Multi-Agents; преимущество последовательной записи в описанных схемах.

[8] [OpenAI Agents SDK. Guardrails](#)

Блокирующий и параллельный режимы; побочные эффекты до завершения проверки; границы охвата инструментов.

[9] [Avižienis A., Laprie J.-C., Randell B., Landwehr C. Basic Concepts and Taxonomy of Dependable and Secure Computing. 2004](#)

Первичная публикация о понятиях fault, error, failure и методах обеспечения надёжности. IEEE TDSC, 1(1), 11-33.

[10] [PostgreSQL 18. Write-Ahead Logging \(WAL\)](#)

Порядок предзаписи и восстановление данных по журналу.

[11] [Anthropic. Demystifying evals for AI agents](#)

Разработка наборов оценки, виды оценщиков, проверка конечного результата и калибровка.

[12] [LangGraph. Persistence](#)

Контрольные точки и хранилища; различие памяти процесса и устойчивого сохранения.

Оригинальный материал Антона предоставлен пользователем в сообщении. Он воспроизведён с указанными редакционными сокращениями; внешние источники относятся к пояснениям и не подменяют авторство исходника.

Контакты в Telegram

Отсканируйте QR-код или нажмите на адрес.

Anton

@EQPRG

Автор оригинальных текстов,
широко известный
в узких кругах.

<https://t.me/EQPRG>



Наш чат

@cinicaichat

Здесь появились посты Антона,
которые затем были собраны
в эту книгу.

<https://t.me/cinicaichat>

